



INSTITUCION:
INSTITUTO SUPERIOR TECNOLÓGICO DE TURISMO Y PATRIMONIO YAVIRAC

CARRERA:
DESARROLLO DE SOFTWARE

ESTUDIANTE:
ALEX ZARUMA
KEVIN SANTANA

DOCENTE:
Ing. BYRON MORENO

ASIGNATURA:
DEVOPS

CURSO:
QUINTO DS "A"

TEMA:
Trabajo Colaborativo en Parejas GitHub



1. INVESTIGACIÓN

Control de Versiones

Definición:

Es un sistema que permite registrar y controlar los cambios realizados en archivos de un proyecto.

Utilidad:

Permite recuperar versiones anteriores, trabajar en equipo y mantener un historial de modificaciones.

Ejemplo:

Dos desarrolladores pueden modificar diferentes partes de una aplicación sin sobrescribir el trabajo del otro.

Fuente

<https://git-scm.com/book/es/v2/Inicio---Sobre-el-Control-de-Versiones-Acerca-del-Control-de-Versiones>

Git

Definición:

Es un sistema de control de versiones distribuido creado por Linus Torvalds.

Utilidad:

Permite gestionar cambios en proyectos de software.

Ejemplo:

Registrar modificaciones mediante commits.

Fuente

<https://git-scm.com/>

GitHub

Definición:

Plataforma en línea para alojar repositorios Git.

Utilidad:

Facilita la colaboración entre desarrolladores.

Ejemplo:

Compartir código mediante repositorios remotos.

Fuente

<https://git-scm.com/book/es/v2/GitHub-Creaci%C3%B3n-y-configuraci%C3%B3n-de-la-cuenta>

Repositorio Local

Definición:

Es la copia del proyecto almacenada en el computador del desarrollador.

Utilidad:

Permite trabajar sin conexión.

Ejemplo:

Carpeta creada mediante:

```
</> Bash  
git init
```

Fuente

<https://docs.github.com/es/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github>



Repositorio Remoto

Definición:

Repositorio almacenado en un servidor como GitHub.

Utilidad:

Compartir código con otros desarrolladores.

Ejemplo:

```
</> Bash  
git push origin main
```

Commit

Definición:

Registro de cambios realizados en el proyecto.

Utilidad:

Guardar versiones del trabajo.

Ejemplo:

```
</> Bash  
git commit -m "Se agregó página de contacto"
```

Branch (Rama)

Definición:

Línea independiente de desarrollo.

Utilidad:

Permite trabajar sin afectar la rama principal.

Ejemplo:

```
</> Bash  
git checkout -b feature-login
```

Merge

Definición:

Proceso de combinar cambios de dos ramas.

Utilidad:

Integrar nuevas funcionalidades.

Ejemplo:

```
</> Bash  
git merge feature-login
```

Pull Request

Definición:

Solicitud para integrar cambios de una rama a otra.

Utilidad:

Permite revisar código antes de fusionarlo.

Ejemplo:

Un colaborador solicita integrar la rama "feature-contacto" a "main".

Fork

Definición:

Copia de un repositorio en una cuenta diferente.

Utilidad:

Permite contribuir a proyectos externos.

Ejemplo:

Copiar un proyecto Open Source para realizar mejoras.

Conflictos de Integración

Definición:

Ocurren cuando dos personas modifican la misma línea de código.

Utilidad:

Obligan a decidir qué cambios conservar.

```
<<<<<<< HEAD
Hola Mundo
=====
Hola GitHub
>>>>>>> rama-feature
```

Integración Continua (CI)

Definición:

Práctica que ejecuta pruebas automáticamente cuando se realizan cambios.

Utilidad:

Detectar errores tempranamente.

Ejemplo:

GitHub Actions ejecutando pruebas tras cada push.

Entrega Continua (Continuous Delivery)

Definición:

Preparar automáticamente una aplicación para su despliegue.

Utilidad:

Reducir tiempo de publicación.

Ejemplo:

Generar automáticamente paquetes listos para producción.

Despliegue Continuo (Continuous Deployment)

Definición:

Publicar automáticamente cambios en producción.

Utilidad:

Automatizar completamente el despliegue.

Ejemplo:

Actualizar una página web después de cada merge.



GitHub Actions

Definición:

Herramienta de automatización integrada en GitHub.

Utilidad:

Ejecutar pruebas, compilaciones y despliegues.

Ejemplo:

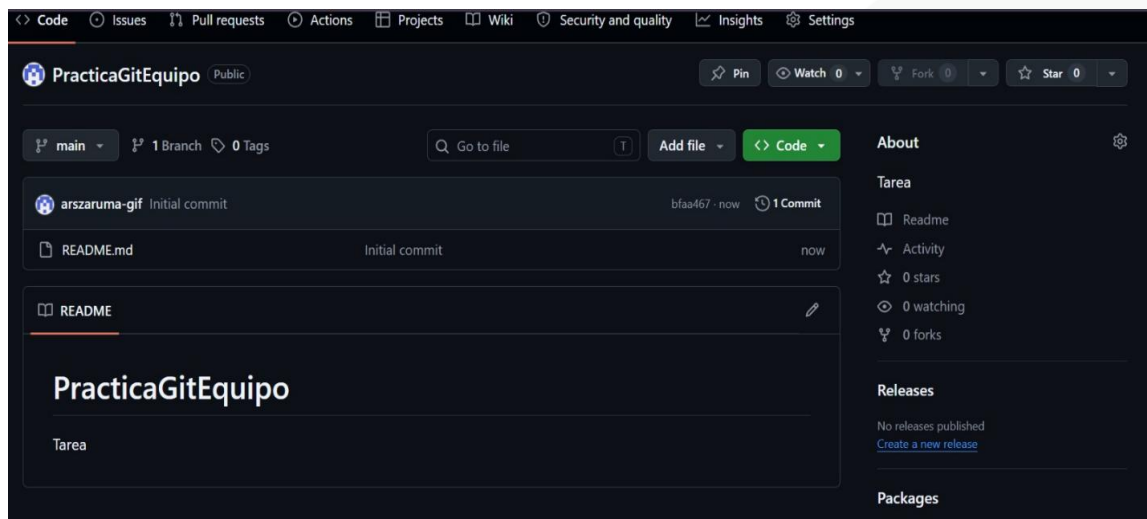
Workflow que verifica un proyecto HTML después de cada push.

Fuente:

<https://github.com/features/actions>

2. Demostración Práctica

2.1 Creación del repositorio en GitHub



Creamos el repositorio denominado **PracticaGitEquipo** en GitHub con el objetivo de desarrollar la práctica de control de versiones de forma colaborativa. Durante este proceso configuramos el repositorio e inicializamos el proyecto con un archivo README.

Resultado obtenido: El repositorio fue creado exitosamente y quedó disponible para el trabajo conjunto de ambos integrantes.

2.2 Creación de ramas de trabajo por cada integrante

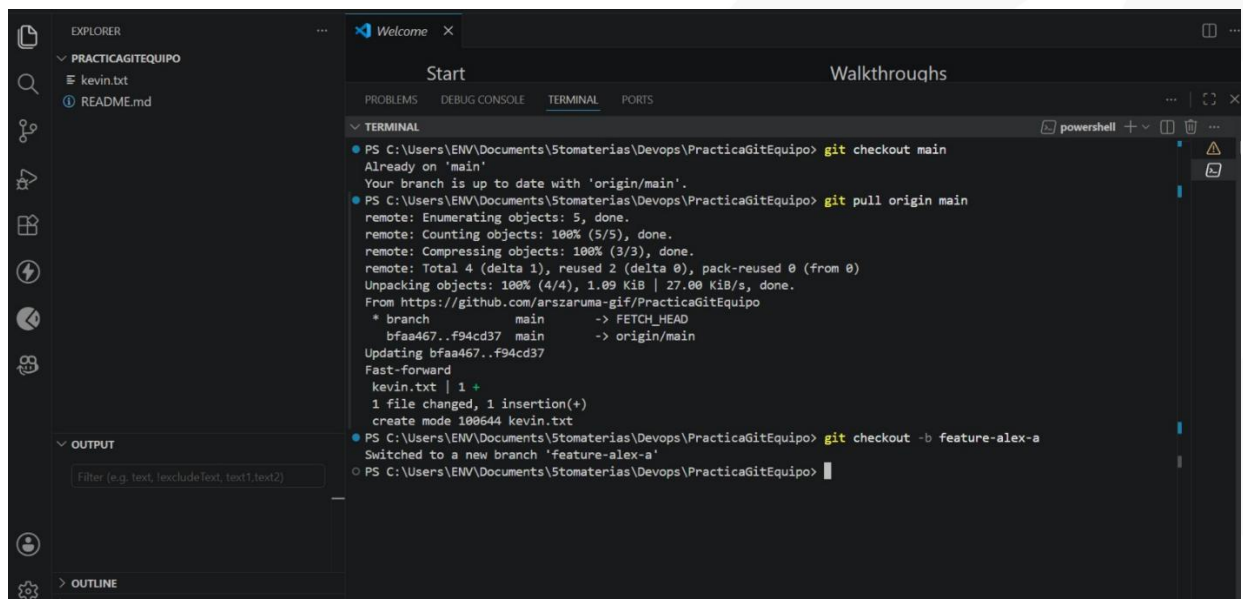
Rama del Estudiante B

```
PS C:\Users\User> cd PracticaGitEquipo
PS C:\Users\User\PracticaGitEquipo> git checkout -b feature-kevin-b
Switched to a new branch 'feature-kevin-b'
PS C:\Users\User\PracticaGitEquipo> git branch
* feature-kevin-b
  main
PS C:\Users\User\PracticaGitEquipo> 
```

Creé la rama **feature-kevin-b** para desarrollar mis cambios sin afectar la rama principal del proyecto.

Resultado obtenido: Se generó una rama independiente para trabajar de manera segura y organizada.

Captura 3: Rama del Estudiante A

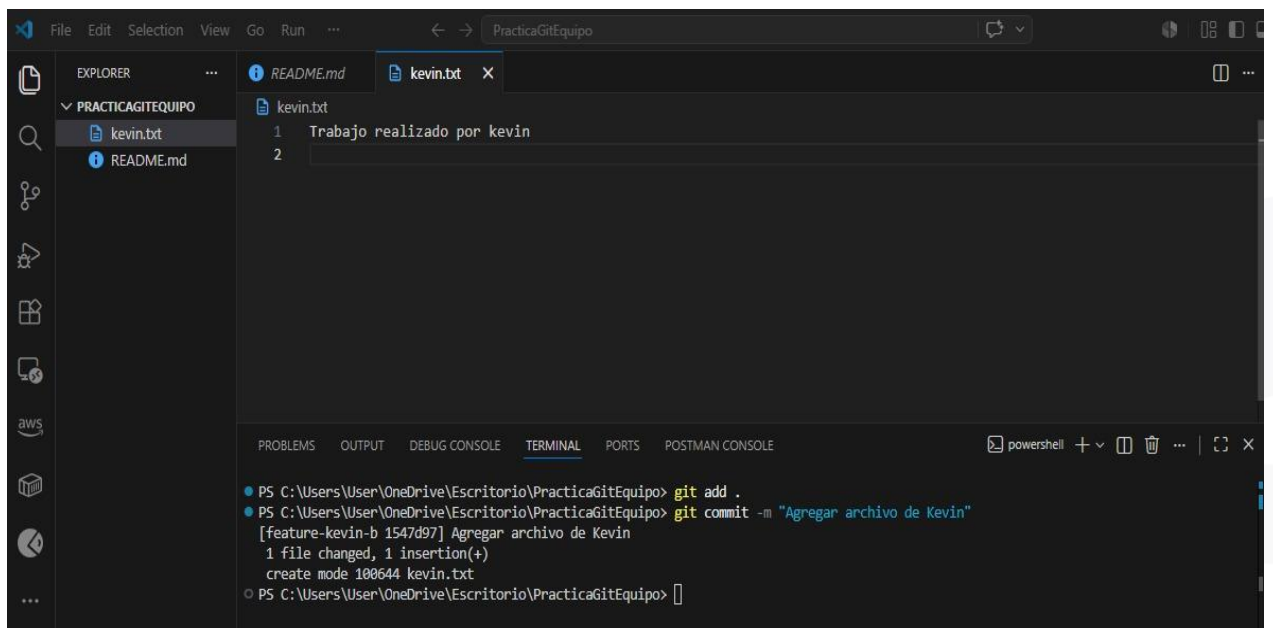


Mi compañero creó la rama **feature-alex-a** para implementar sus cambios de forma independiente.

Resultado obtenido: Ambos integrantes contábamos con ramas separadas para desarrollar nuestras actividades.

2.3 Commits realizados por ambos integrantes

Captura 4: Commit del Estudiante B



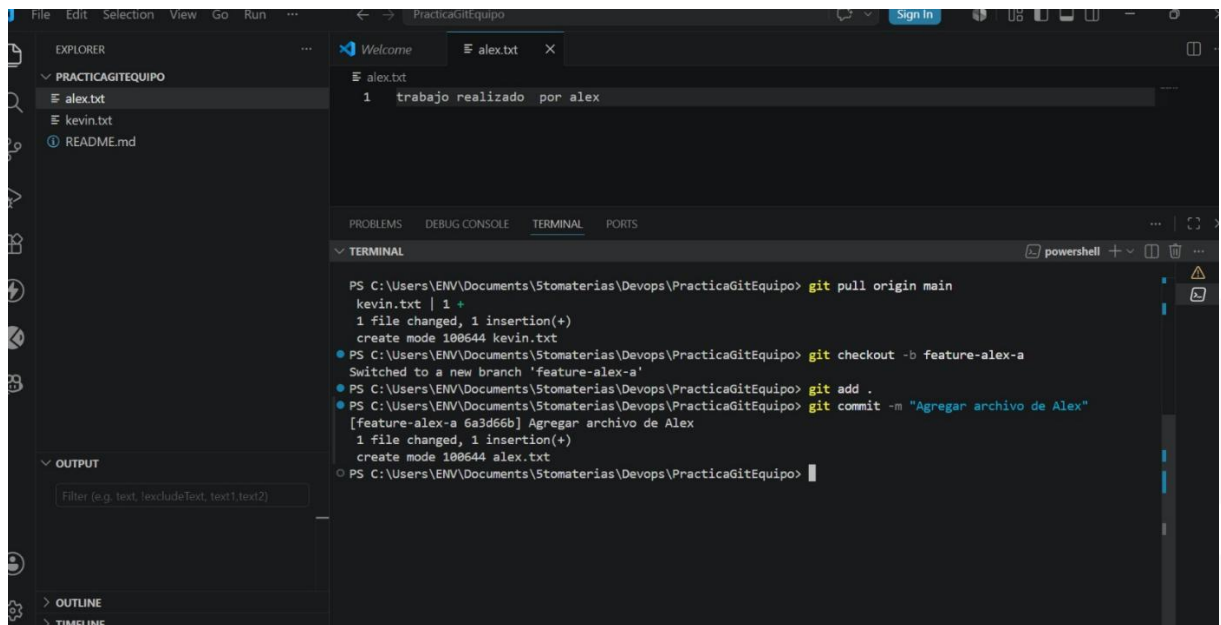
The screenshot shows the Visual Studio Code interface with a file explorer on the left showing a project named 'PRACTICAGITEQUIPO' containing 'kevin.txt' and 'README.md'. The editor window shows 'kevin.txt' with two lines of text: '1 Trabajo realizado por kevin' and '2'. The terminal at the bottom shows the following commands and output:

```
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo> git add .
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo> git commit -m "Agregar archivo de Kevin"
[feature-kevin-b 1547d97] Agregar archivo de Kevin
1 file changed, 1 insertion(+)
create mode 100644 kevin.txt
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo>
```

Realicé un commit después de crear y modificar el archivo correspondiente a mi trabajo.

Resultado obtenido: Los cambios quedaron registrados en el historial local del repositorio.

Captura 5: Commit del Estudiante A

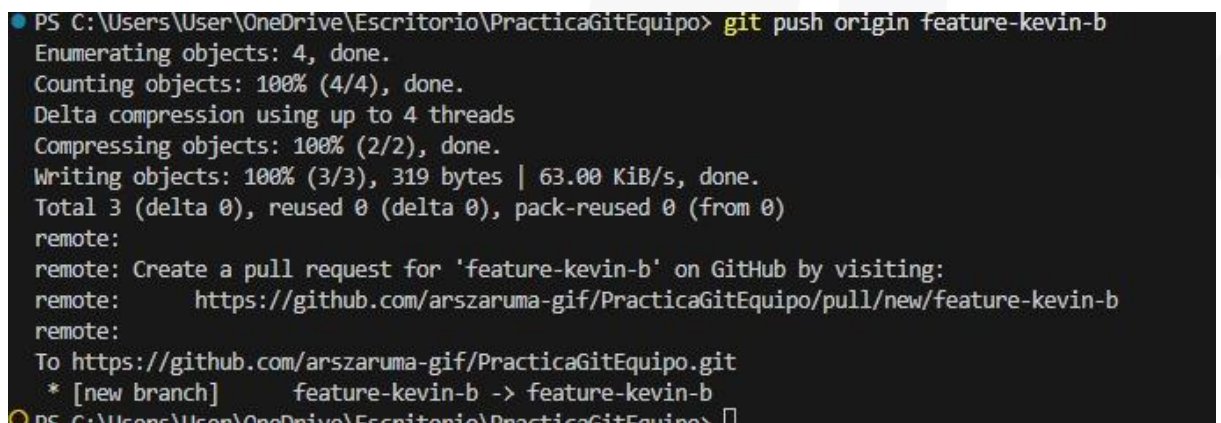


```
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git pull origin main
kevin.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 kevin.txt
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git checkout -b feature-alex-a
Switched to a new branch 'feature-alex-a'
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git add .
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git commit -m "Agregar archivo de Alex"
[feature-alex-a 6a3d66b] Agregar archivo de Alex
1 file changed, 1 insertion(+)
create mode 100644 alex.txt
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo>
```

Mi compañero realizó un commit con las modificaciones efectuadas en su rama de trabajo.
Resultado obtenido: Los cambios quedaron almacenados correctamente en Git.

2.4 Publicación de cambios al repositorio remoto

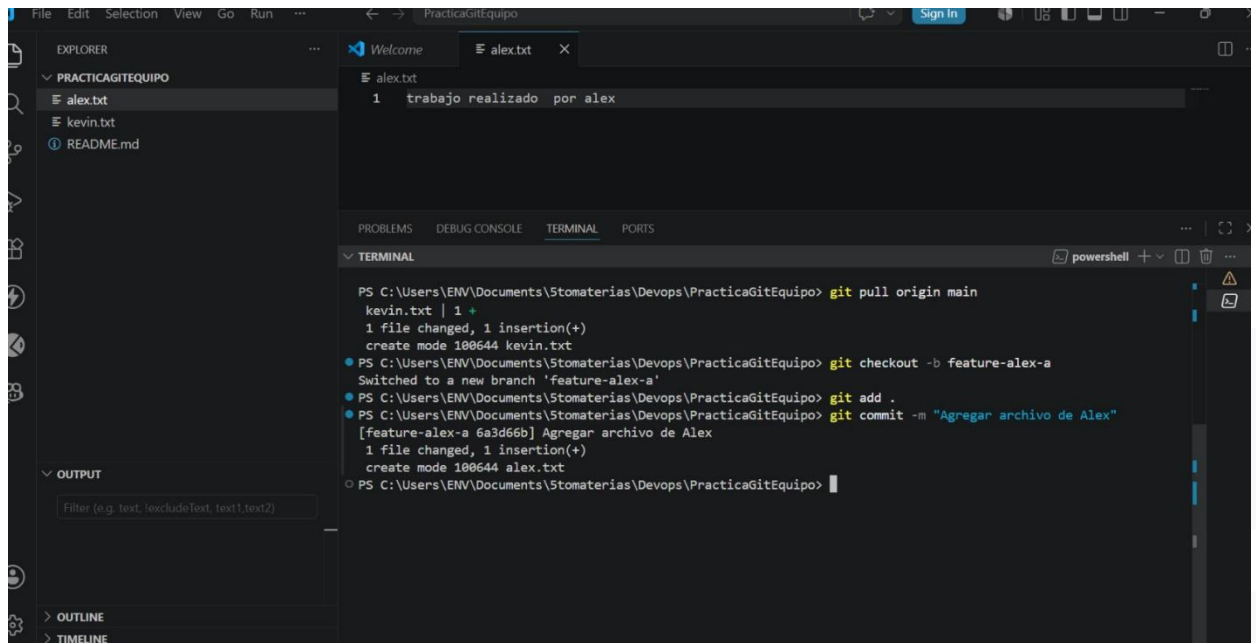
Captura 6: Publicación de cambios del Estudiante B



```
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo> git push origin feature-kevin-b
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 4 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 319 bytes | 63.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
remote:
remote: Create a pull request for 'feature-kevin-b' on GitHub by visiting:
remote:   https://github.com/arszaruma-gif/PracticaGitEquipo/pull/new/feature-kevin-b
remote:
To https://github.com/arszaruma-gif/PracticaGitEquipo.git
 * [new branch]   feature-kevin-b -> feature-kevin-b
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo>
```

Publiqué mi rama en GitHub mediante el comando push.
Resultado obtenido: Los cambios quedaron disponibles en el repositorio remoto.

Captura 7: Publicación de cambios del Estudiante A



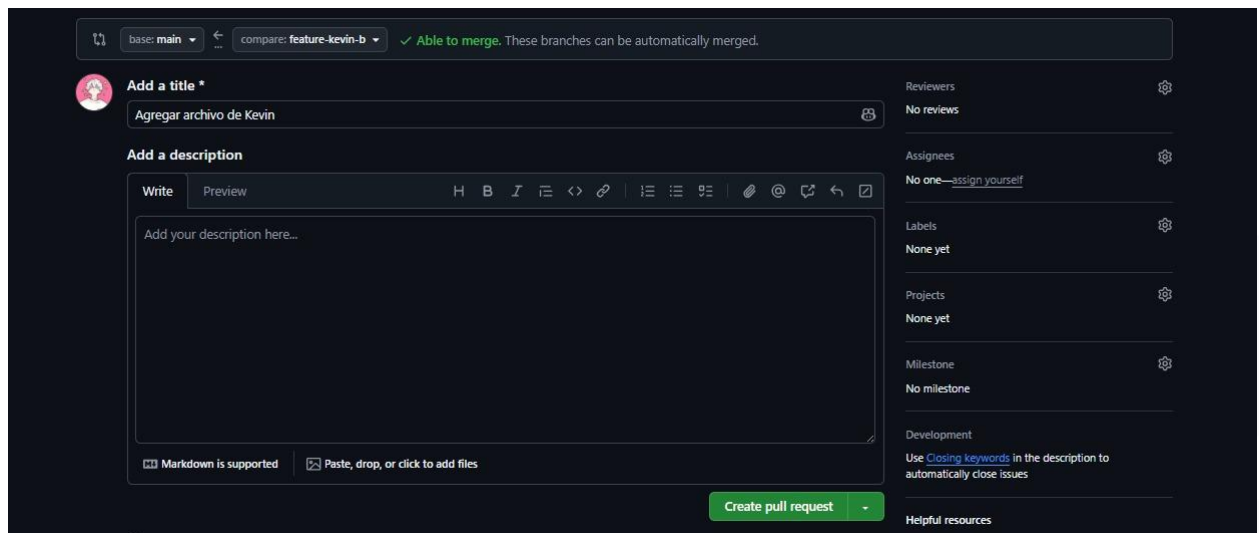
```
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git pull origin main
kevin.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 kevin.txt
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git checkout -b feature-alex-a
Switched to a new branch 'feature-alex-a'
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git add .
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo> git commit -m "Agregar archivo de Alex"
[feature-alex-a 6a3d66b] Agregar archivo de Alex
1 file changed, 1 insertion(+)
create mode 100644 alex.txt
PS C:\Users\ENV\Documents\Stomaterias\Devops\PracticaGitEquipo>
```

Mi compañero publicó su rama de trabajo en GitHub.

Resultado obtenido: La rama quedó disponible para crear un Pull Request.

2.5 Creación de Pull Requests

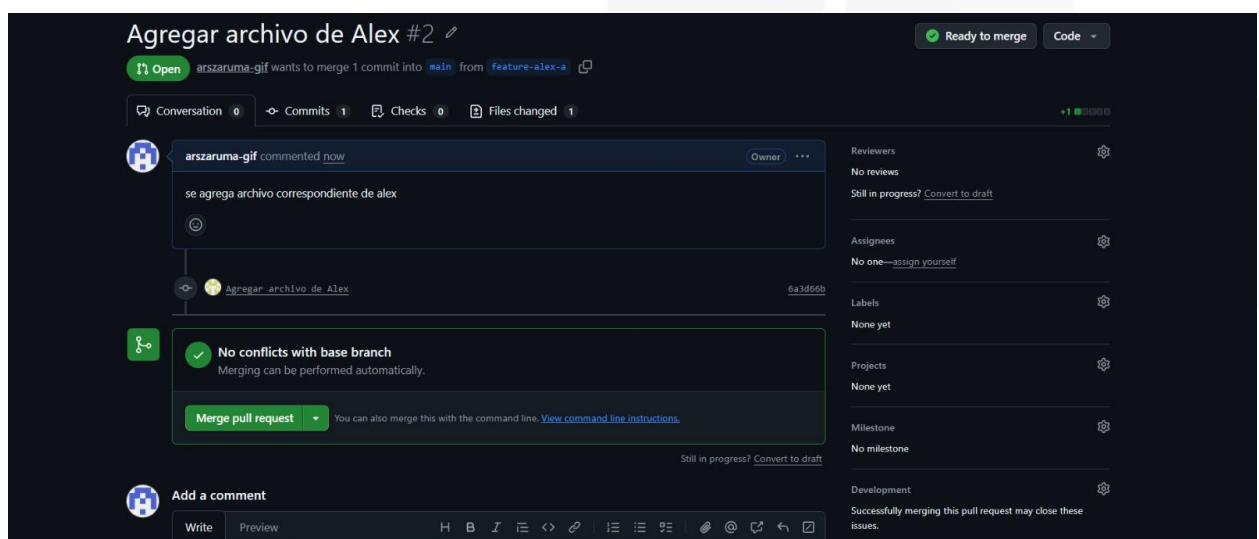
Captura 8: Pull Request del Estudiante B



Creé un Pull Request para solicitar la integración de mis cambios desde la rama feature-kevin-b hacia la rama principal.

Resultado obtenido: Los cambios quedaron listos para ser revisados antes de la integración.

Captura 9: Pull Request del Estudiante A

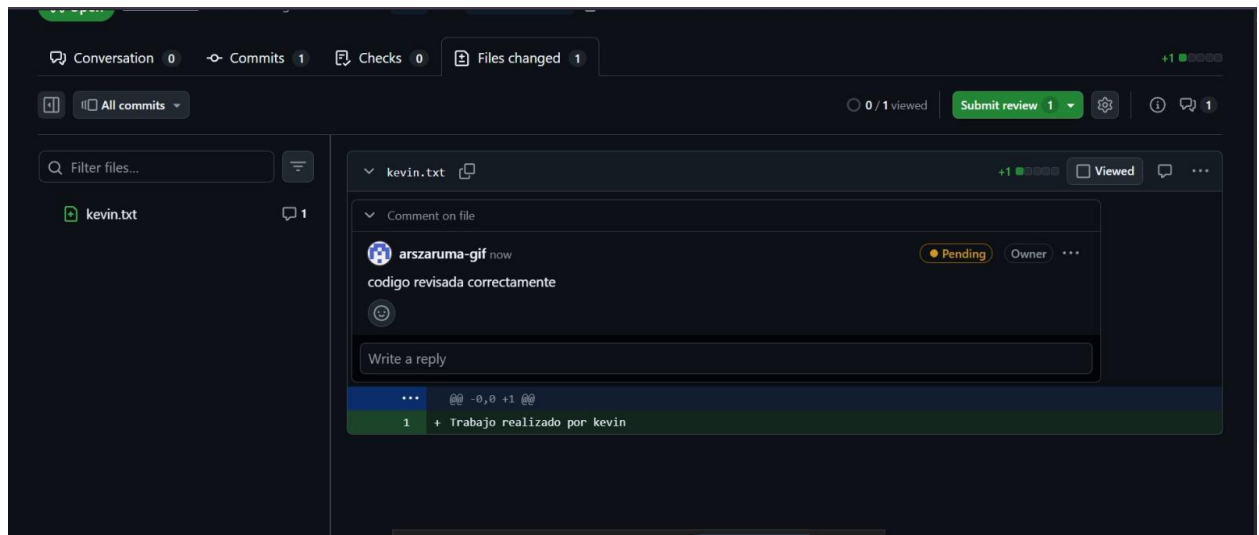


Mi compañero creó un Pull Request desde la rama feature-alex-a hacia main.

Resultado obtenido: Los cambios quedaron preparados para el proceso de revisión.

2.6 Revisión de código

Captura 10: Revisión de código

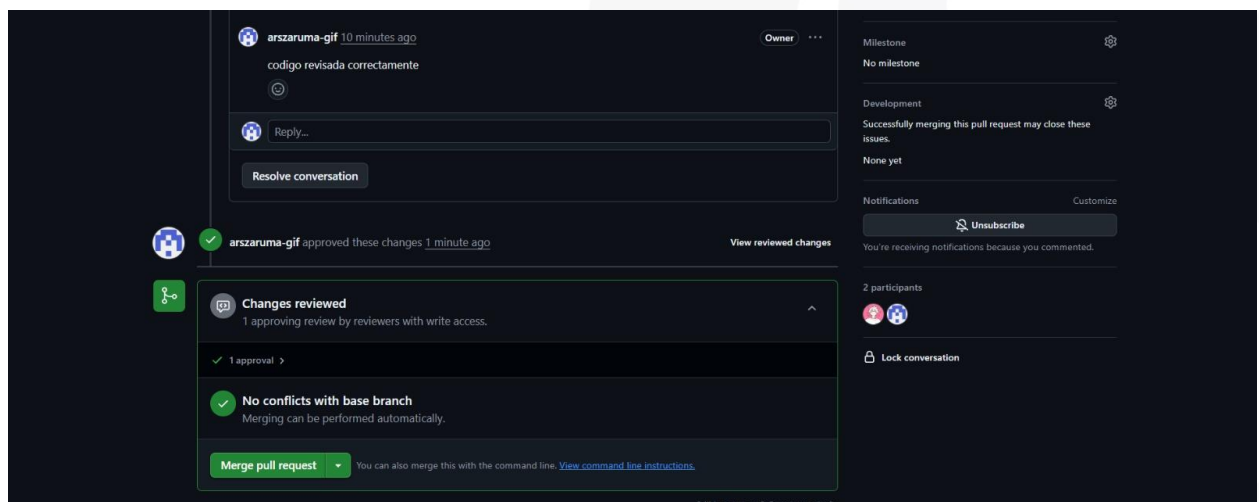


Revisamos los cambios realizados en los Pull Requests para verificar que el código fuera correcto y cumpliera con los objetivos de la práctica.

Resultado obtenido: Los cambios fueron validados satisfactoriamente.

2.7 Aprobación de Pull Requests

Captura 11: Aprobación de Pull Request

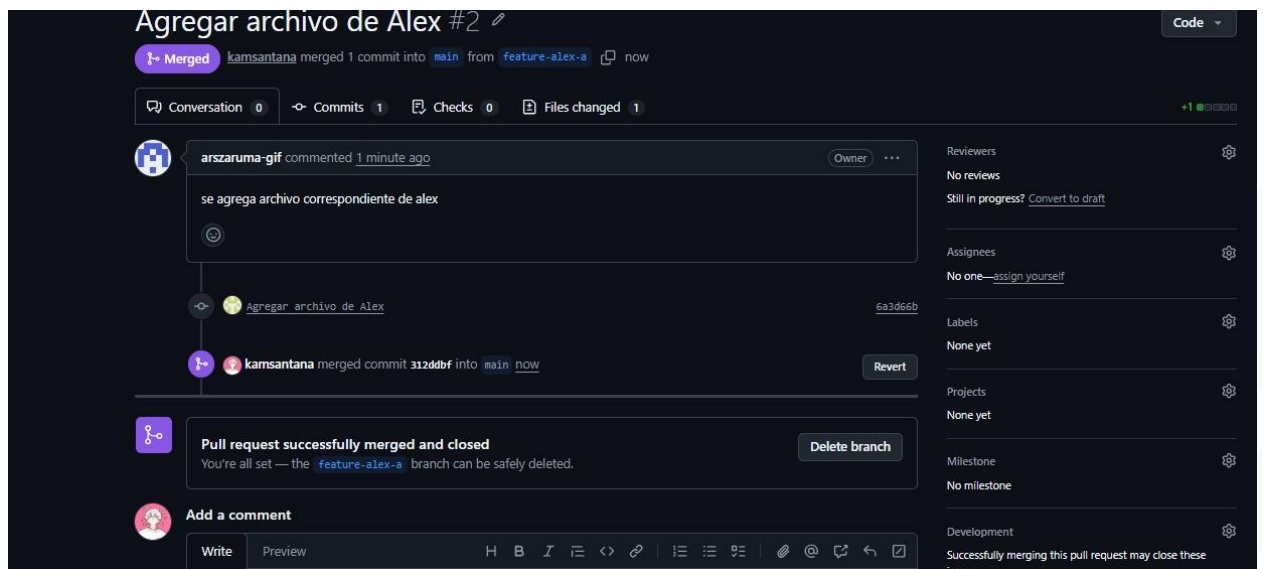


Aprobamos los Pull Requests utilizando la opción correspondiente de GitHub después de finalizar la revisión.

Resultado obtenido: Los cambios quedaron autorizados para ser fusionados con la rama principal.

2.8 Ejecución del proceso de Merge

Captura 12: Merge realizado

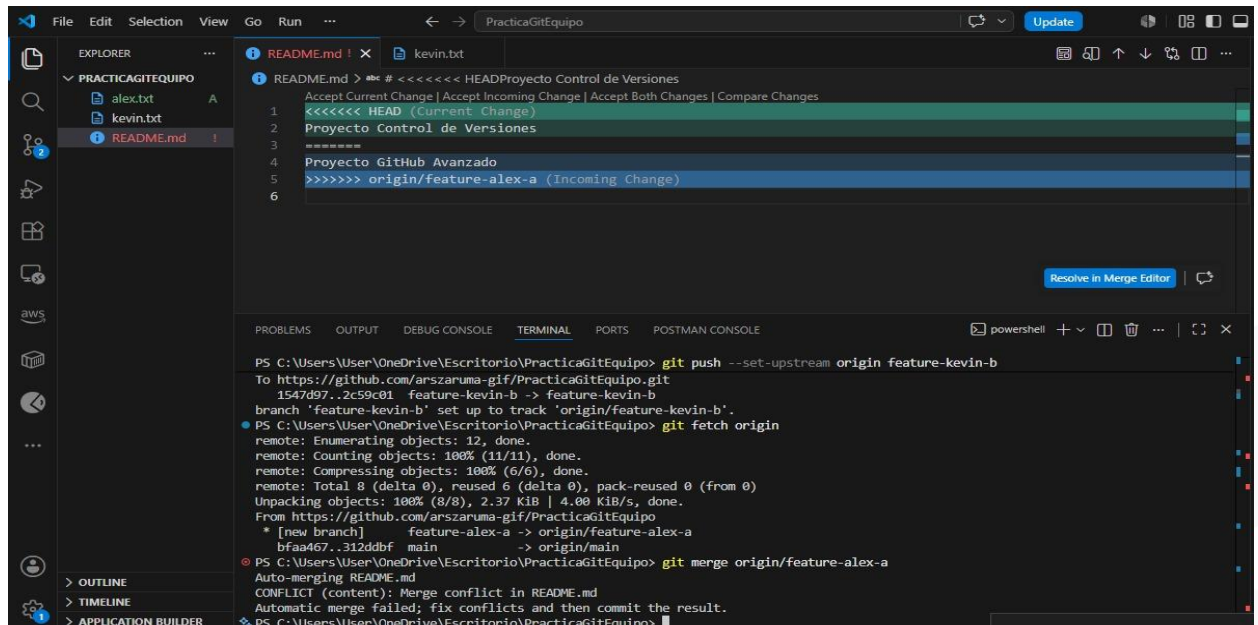


Realizamos el proceso de Merge para integrar los cambios de las ramas de trabajo a la rama principal del repositorio.

Resultado obtenido: Los cambios fueron incorporados correctamente al proyecto.

2.9 Generación de conflictos de integración

Captura 13: Conflicto detectado



```

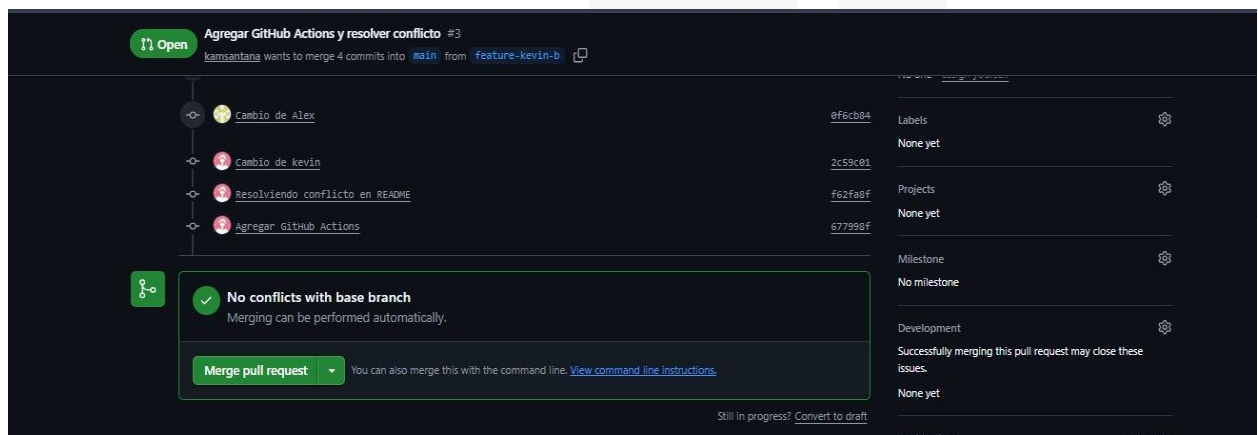
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo> git push --set-upstream origin feature-kevin-b
To https://github.com/arszaruma-gif/PracticaGitEquipo.git
1547d97..2c59c01 feature-kevin-b -> feature-kevin-b
branch 'feature-kevin-b' set up to track 'origin/feature-kevin-b'.
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo> git fetch origin
remote: Enumerating objects: 12, done.
remote: Counting objects: 100% (11/11), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 8 (delta 0), reused 6 (delta 0), pack-reused 0 (from 0)
Unpacking objects: 100% (8/8), 2.37 KiB | 4.00 KiB/s, done.
From https://github.com/arszaruma-gif/PracticaGitEquipo
* [new branch] feature-alex-a -> origin/feature-alex-a
* bfaa467..312ddb main -> origin/main
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo> git merge origin/feature-alex-a
Auto-merging README.md
CONFLICT (content): Merge conflict in README.md
Automatic merge failed; fix conflicts and then commit the result.
PS C:\Users\User\OneDrive\Escritorio\PracticaGitEquipo>
  
```

Generamos un conflicto de integración modificando la misma línea del archivo README.md desde diferentes ramas de trabajo.

Resultado obtenido: GitHub detectó el conflicto e impidió realizar la integración automática.

2.10 Resolución de conflictos de integración

Captura 14: Resolución del conflicto

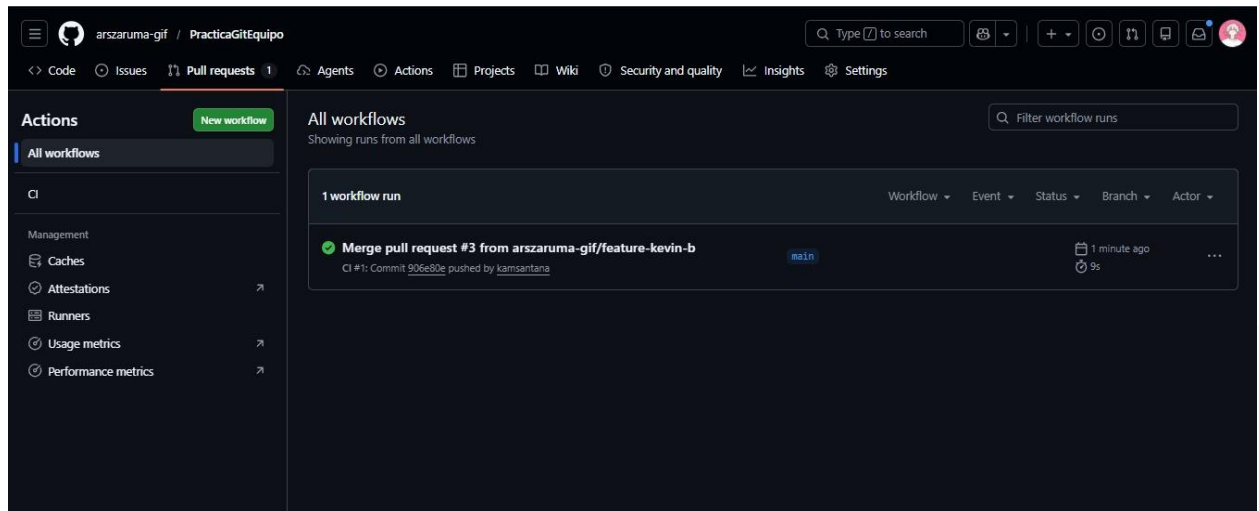


Editamos manualmente el archivo en conflicto y seleccionamos la versión correcta para conservar los cambios necesarios.

Resultado obtenido: El conflicto fue resuelto exitosamente y se completó la integración.

2.11 Ejecución exitosa del workflow de GitHub Actions

Captura 15: Workflow ejecutado correctamente

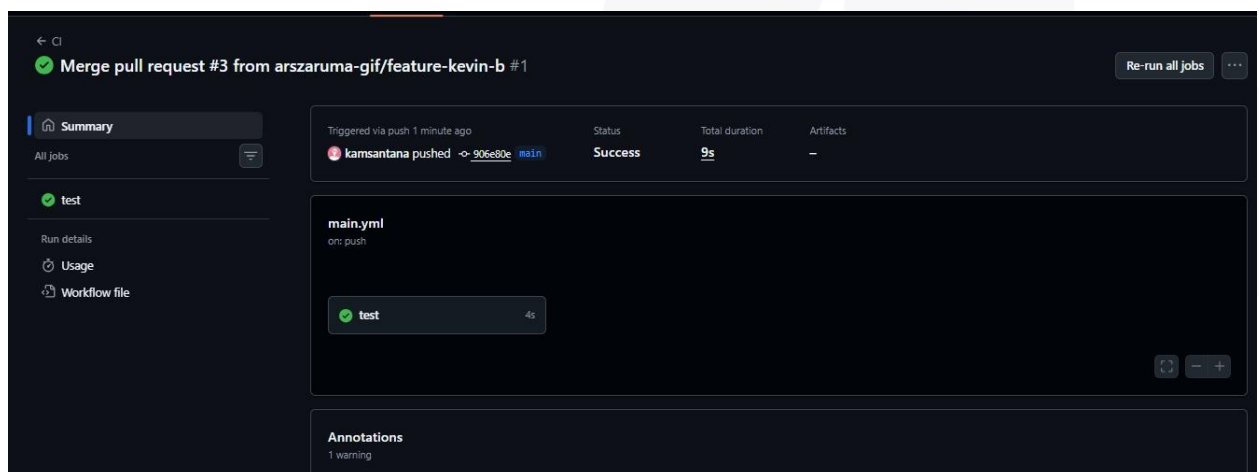


Configuramos un workflow básico utilizando GitHub Actions para automatizar una tarea de validación del repositorio.

Resultado obtenido: El workflow se ejecutó correctamente y mostró un estado exitoso mediante un indicador verde.

2.12 Estado final del repositorio

Captura 16: Estado final del repositorio



Verificamos el estado final del repositorio después de integrar todas las ramas, resolver conflictos y ejecutar el workflow de GitHub Actions.

Resultado obtenido: El repositorio quedó actualizado, organizado y con todos los cambios de ambos integrantes integrados correctamente en la rama principal.

URL del Repositorio GitHub

Repositorio del proyecto:

<https://github.com/arszaruma-gif/PracticaGitEquipo.git>

Conclusiones Individuales

▪ Conclusión de Alex Zaruma

Durante el desarrollo de esta práctica adquirí experiencia en el uso de Git y GitHub como herramientas de control de versiones. Aprendí a crear ramas de trabajo, realizar commits y publicar cambios en un repositorio remoto. También comprendí la importancia de los Pull Requests para la revisión y aprobación de código antes de integrar cambios al proyecto principal. La resolución de conflictos de integración me permitió entender cómo manejar situaciones comunes en el trabajo colaborativo. Finalmente, la implementación de GitHub Actions me ayudó a conocer conceptos básicos de automatización e integración continua.

▪ Conclusion de Kevin Santana

La práctica permitió fortalecer los conocimientos relacionados con el trabajo colaborativo utilizando Git y GitHub. Se comprendió la importancia de mantener un historial organizado mediante commits descriptivos y el uso de ramas independientes para el desarrollo de nuevas funcionalidades. Además, se aprendió a revisar y aprobar cambios mediante Pull Requests antes de realizar la integración al proyecto principal. La resolución de conflictos contribuyó a mejorar la capacidad de análisis y solución de problemas durante el desarrollo. Finalmente, el uso de GitHub Actions permitió conocer herramientas modernas utilizadas en entornos profesionales de desarrollo de software.

▪ Bibliografía

Git. (2025). *Git Documentation*. Recuperado de <https://git-scm.com/doc>

GitHub. (2025). *GitHub Documentation*. Recuperado de <https://docs.github.com>

Atlassian. (2025). *Git Tutorials and Training*. Recuperado de <https://www.atlassian.com/git>

Chacon, S., & Straub, B. (2014). *Pro Git* (2.^a ed.). Apress. Recuperado de <https://git-scm.com/book/en/v2>

GitHub. (2025). *GitHub Actions Documentation*. Recuperado de <https://docs.github.com/actions>